

MLIR Part 7 - Transformers

Stephen Diehl

transformer architecture



The transformer architecture revolutionized the field of natural language processing when introduced in the landmark 2017 paper [Attention is All You Need](#). Breaking away from traditional sequence models, transformers employ **self-attention** mechanisms (more on this later) as their core building block, enabling them to capture long-range dependencies in data with remarkable efficiency. In essence, the transformer can be viewed as a general-purpose computational substrate—a programmable logical tissue that reconfigures based on training data and can be stacked as layers build large models exhibiting fascinating emergent behaviors.

Each of these layers contains two essential sublayers working in tandem: a **multi-head self-attention mechanism** and a **position-wise feed-forward network**. The multi-head attention allows the model to simultaneously focus on different parts of the input sequence, capturing various relationships between tokens from different representational perspectives. The feed-forward network, applied identically to each position, then introduces non-linearity and expands the model's capacity to learn complex patterns.

Since the attention mechanism processes all tokens in parallel, transformers require an innovative approach to handling sequential information. This challenge is addressed through positional encodings—either learned embeddings or fixed sinusoidal functions—that are added to the token embeddings at the input layer. These encodings provide the model with crucial information about token positions that would otherwise be lost in the parallel processing.

The power of transformers stems from their ability to dynamically weight the importance of different tokens

during processing. Consider the word "bank" in the sentence "I went to the bank to deposit money." When processing this ambiguous word, the model can attend strongly to contextual clues like "deposit" and "money," helping it correctly interpret "bank" as a financial institution rather than a riverside. This contextual awareness enables transformers to excel at tasks requiring nuanced language understanding.

Layer normalization and residual connections further enhance the transformer architecture. Layer normalization stabilizes the learning process by normalizing activations within each layer, while residual connections create shortcuts for gradient flow during training. These components work together to mitigate the vanishing gradient problem that often plagues deep networks, allowing transformers to be trained effectively despite their depth.

Remarkably, researchers found that as they scaled up transformer models by increasing their layer count, embedding dimensions, and training data, these architectures demonstrated remarkable emergent capabilities. Larger models exhibit abilities not present in their smaller counterparts, including few-shot learning, reasoning, and even rudimentary understanding of concepts they weren't explicitly trained on. This scaling effect has driven much of the recent progress in artificial intelligence.

The decoder-only Transformer architecture which we'll build from scratch here, is composed of multiple identical decoder blocks stacked vertically. Unlike the original Transformer which had both encoder and decoder components, modern language models typically use only the decoder portion for next-token prediction tasks. This architecture has proven highly scalable, with models ranging from hundreds of millions to now billions of parameters in frontier models.

Let's walk through a pure NumPy implementation of the GPT-2 model, component by component. It only comes out to about 100 lines of code in total. The full source code is available [on GitHub here](#).

There are several data structures of the neural network weights that we'll download from the internet later. For now just know that they're a collection of arrays of various dimensions.

LinearParams - A linear layer with weights and bias parameters for the linear transformations.

LayerNormParams - A layer normalization layer with gain and bias parameters for normalization.

AttentionParams - A multi-head attention layer containing query, key, value and output projection parameters.

BlockParams - A transformer block containing attention and feed-forward parameters.

ModelParams - The full model parameters including token embeddings, position embeddings, transformer blocks and final layer norm.

HParams - Hyperparameters for the model including the number of layers, heads, embedding dimension, etc.

In our code we'll use the following abbreviations:

g - Gamma (scale parameter for layer normalization)

b - Beta (bias parameter)

w - Weight matrix/array

wte - Word/Token Embeddings

wpe - Word Position Embeddings

ln - Layer Normalization

mlp - Multi-Layer Perceptron

fc - Fully Connected layer

qkv - Query, Key, Value (attention components)

attn - Attention

proj - Projection (linear transformation)

BPE Tokenization

Byte-Pair Encoding (shortened as BPE) is a tokenization algorithm that addresses the challenge of out-of-vocabulary words by breaking words into smaller units called subwords. Initially developed as a data compression technique, BPE was adapted for natural language processing by treating words as sequences of characters. The core idea of BPE is to iteratively merge the most frequent adjacent symbol pairs in a corpus into new subword units. This process begins by splitting each word in the corpus into individual characters, forming the initial vocabulary consisting of all unique characters.

The BPE algorithm then proceeds with **iterative merging**. In each iteration, it counts all adjacent symbol pairs across the corpus. The pair with the highest frequency is then selected and merged into a new symbol or subword. This merge operation is applied to all occurrences of the selected pair in the corpus, effectively removing the space between them in the tokenized representations of the words. The newly formed subword is added to the vocabulary. This iterative process continues until the vocabulary reaches a predefined target size or no more frequent pairs can be found. The sequence of merge operations performed during training is crucial for tokenizing new, unseen words.

Once the BPE tokenizer is trained (i.e., the sequence of merges is determined), it can be used to tokenize new text. When tokenizing a word, the tokenizer first checks if the whole word (with the initial word boundary marker) exists in the vocabulary. If not, it splits the word into characters and iteratively applies the learned merge rules in the order they were learned. Any character not present in the initial character set might be replaced by an unknown token (e.g., <UNK>). BPE is widely used in modern language models and is particularly useful for handling rare words and languages without clear word boundaries like Chinese, where initial tokenization might start with individual characters.

We're not going to implement BPE from scratch, but you can find a simple Python implementation [here](#). Instead we'll just load the existing tokenizer from the original GPT-2 repo.

```
encoder = get_encoder("", "model")

# encode a string into a list of tokens
tokens = encoder.encode("Hello world")

print(tokens) # [15496, 995]
```

Here we see the tokenizer has encoded the string "Hello world" into a list of tokens [15496, 995] which happens to have two words and two tokens. However the string "Barack Obama" is encoded into [10374, 441, 2486] which has two words and three tokens.

Tokenization and Position Embeddings

From the string input we first tokenize it into a list of tokens and then we look up the token embeddings for each token in the `params.wte` matrix.

```
x = params.wte[inputs] + params.wpe[range(len(inputs))]
```

This line is crucial for preparing the input sequence before it gets processed by the main transformer blocks. It combines two types of information for each token in the input (the position and the token itself). In the model configuration there are two large embedding matrices the *token* embedding matrix and the *position* embedding matrix.

Token Embeddings (`params.wte[inputs]`):

The `inputs` parameter represents your sequence of input tokens as a list of integers (token IDs). For example, if your input text "Hello world" is tokenized into `[15496, 995]`, then `inputs` would be `[15496, 995]`.

The `params.wte` refers to the **W**ord **T**oken **E**mbding matrix. It's essentially a large table (lookup table) where each row corresponds to a unique token ID in the model's vocabulary, and the row itself is a vector (e.g., of size 768 for the small GPT-2 model). This vector is the learned numerical representation, or "embedding," for that specific token.

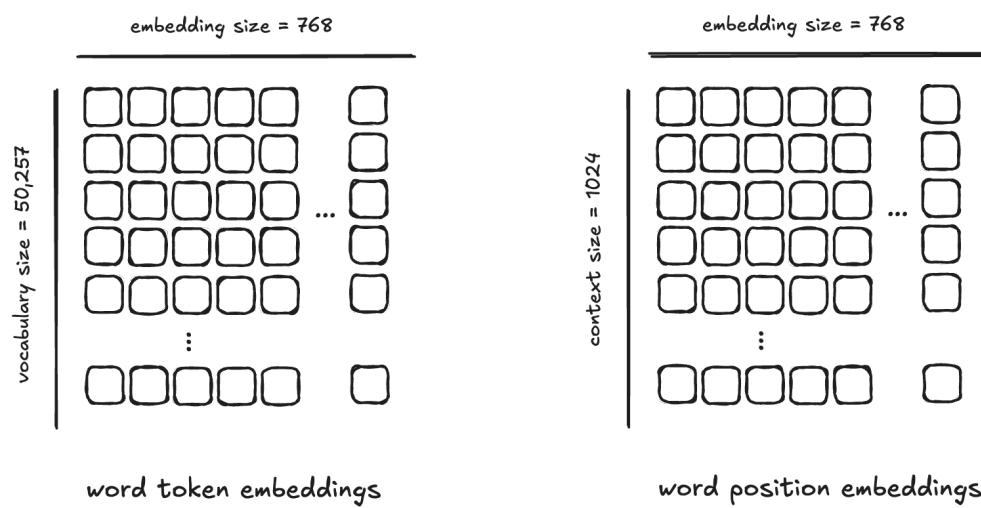
The `params.wte[inputs]` operation performs a lookup. It takes each integer ID in your `inputs` list and retrieves the corresponding embedding vector from the `params.wte` matrix. The result is a matrix where each row is the token embedding for the corresponding token in your input sequence. So, for `[15496, 995]`, you'd get a matrix with two rows: the embedding for token 15496 and the embedding for token 995.

Position Embeddings (`params.wpe[range(len(inputs))]`):

Transformers process tokens in parallel using self-attention, which doesn't inherently know the order of tokens. To provide this sequential information, we use position embeddings. The `len(inputs)` function calculates the length of your input sequence (e.g., 2 for `[15496, 995]`). Using `range(len(inputs))` generates a sequence of position indices starting from 0: `[0, 1]` in our example.

The `params.wpe` refers to the **W**ord **P**osition **E**mbding matrix. Similar to `wte`, it's a lookup table, but its rows correspond to positions in a sequence (0, 1, 2, ... up to the maximum sequence length the model can handle). Each row is a learned vector representing that specific position. GPT-2 uses *learned* positional embeddings, meaning these vectors are parameters learned during training.

Finally, `params.wpe[range(len(inputs))]` performs a lookup using the position indices. It retrieves the embedding vector for position 0, then position 1, and so on, for the length of your input. The result is a matrix where each row is the position embedding for the corresponding position in the input sequence.

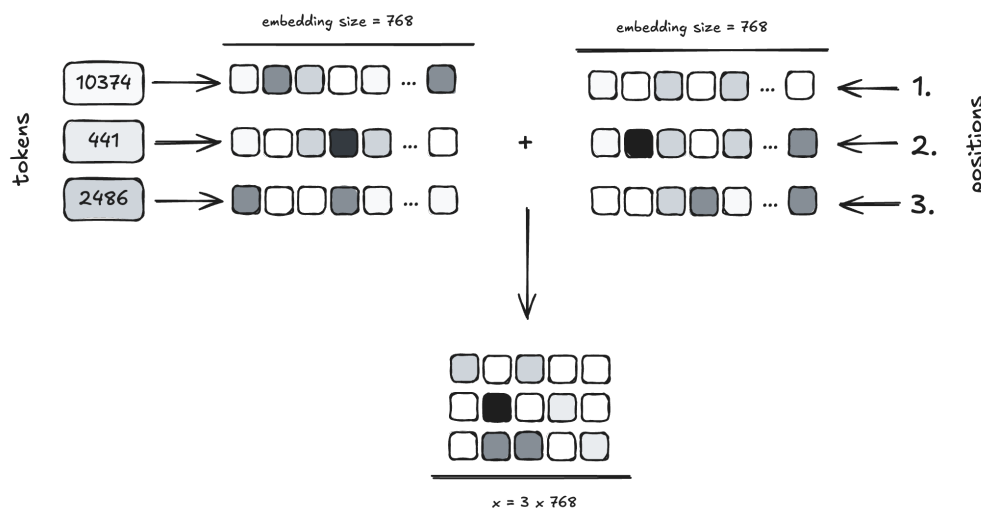


Addition (+):

The token embedding matrix (representing *what* each token is) and the position embedding matrix (representing *where* each token is in the sequence) are simply added together element-wise. Both matrices have the same shape (sequence length $\times$ embedding dimension).

The resulting matrix x now contains vectors where each vector encodes both the token's identity and its position. This combined representation is the final input that gets fed into the stack of transformer blocks for processing.

In summary, this line `x = params.wte[inputs] + params.wpe[range(len(inputs))]` creates the initial representation for the input sequence by looking up the learned vector for each token and adding the learned vector for its position. This allows the subsequent transformer layers to process the tokens while being aware of their order.



The size of the matrix is `[sequence_length, embedding_dimension]`. We denote sequence length as N in the code.

GELU

GELU (Gaussian Error Linear Unit) is an activation function used in the Transformer architecture, particularly within the feed-forward networks (FFN) of each block. Its primary role is to introduce non-linearity into the model, allowing it to learn more complex patterns and relationships in the data. Without non-linear activation functions, stacking multiple layers would be equivalent to a single linear transformation, severely limiting the model's representational power. In math it's defined as:

$$\text{GELU}(x) = 0.5 \cdot x \cdot \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$$

where `erf` is the [Guass error function](#). In practice we'll use an approximation where we use the `tanh` function to approximate the error function so we don't have to compute integrals involved in the error function. In Python this approximation is implemented as:

```
# gelu:
#   x : (N, 768)
#   out : (N, 768)
def gelu(x: np.ndarray) -> np.ndarray:
    return 0.5 * x * (1 + np.tanh(np.sqrt(2 / np.pi) * (x + 0.044715 * x**3)))
```

Note that this is a scalar function that is applied element-wise to the input.

Softmax

While GELU handles non-linearity within feed-forward networks, the transformer architecture also needs a way to convert raw scores into meaningful probability distributions. This is where the softmax function comes into play. Softmax is a crucial function used primarily in two places within the Transformer architecture: within the attention mechanism and at the final output layer. Its purpose is to convert a vector of raw scores (logits) into a probability distribution. It takes an input vector and exponentiates each element, making them all positive, and then normalizes these values by dividing by their sum. The result is a vector where all elements are between 0 and 1, and they all sum up to 1.

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

In the attention mechanism (`attention` function), softmax is applied to the scaled dot-product scores between queries and keys. This transforms the raw similarity scores into attention weights, which represent the probability distribution over the value vectors – indicating how much focus each value should receive.

At the final layer of the model (just before generating the next token), softmax is typically applied to the logits produced by the final linear projection. This converts the model's raw output scores for each word in the vocabulary into probabilities, allowing us to interpret the scores and sample the next token based on this distribution.

```
# softmax:
#   x : (N, 64)
#   out : (N, 64)
def softmax(x: np.ndarray) -> np.ndarray:
    exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

Layer Normalization

Beyond activation functions and probability conversions, neural networks need methods to stabilize training, especially in deep architectures like transformers. Layer Normalization (LayerNorm) serves this critical purpose. Unlike Batch Normalization which normalizes across the batch dimension, Layer Normalization normalizes the inputs across the features (embedding dimension) for *each* data sample independently.

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2}} + \beta$$

Where μ is the mean and σ^2 is the variance of the input, and γ and β are learnable gain and bias parameters.

In a Transformer block, LayerNorm is typically applied *before* the multi-head attention sub-layer and *before* the feed-forward network sub-layer (this is known as pre-LN). It works by calculating the mean and variance of all the activations within a single layer for a single training example. It then normalizes the activations using this mean and variance. Crucially, it also introduces learnable gain (g) and bias (b) parameters, allowing the network to scale and shift the normalized output, preserving its representational capacity.

LayerNorm helps mitigate issues like vanishing or exploding gradients, allows for higher learning rates, and makes the model less sensitive to the scale of the parameters and initialization. By normalizing the activations within each layer, it ensures that the inputs to the subsequent layers are consistently scaled, leading to smoother and more stable training dynamics.

```
# layer_norm:
#   x : (N, 768)
#   g : (768,)
#   b : (768,)
#   out : (N, 768)
def layer_norm(
    x: np.ndarray, g: np.ndarray, b: np.ndarray, eps: float = 1e-5
) -> np.ndarray:
    mean = np.mean(x, axis=-1, keepdims=True)
    variance = np.var(x, axis=-1, keepdims=True)
    return g * (x - mean) / np.sqrt(variance + eps) + b
```

Linear Projection

With normalization in place to stabilize the network, transformers need a way to transform representations from one space to another. This is where linear layers come in. The linear layer, also known as a fully connected or dense layer, is one of the most fundamental components in neural networks. It performs a linear transformation on the input data.

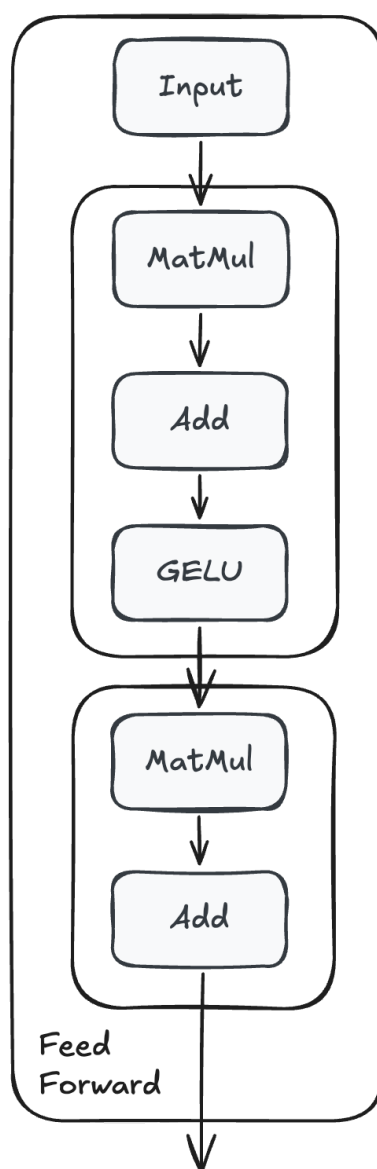
Mathematically, it computes $\text{output} = \text{input} @ \text{weights} + \text{bias}$, where $@$ represents matrix multiplication. The `input` is the data fed into the layer, `weights` is a matrix of learnable parameters, and `bias` is a learnable vector that gets added to the result. The weights matrix effectively projects the input vector from its original dimension to a new dimension (defined by the shape of the weights matrix), and the bias allows for shifting the output.

```
# linear:
#   x : (N, 768)
#   w : (768, 3072)
#   b : (3072,)
#   out : (N, 3072)
def linear(x: np.ndarray, w: np.ndarray, b: np.ndarray) -> np.ndarray:
    return x @ w + b
```

Feed-Forward Network

Having covered the individual building blocks of normalization, activation, and linear transformation, we can now examine how these components come together in the Feed-Forward Network (FFN). The FFN, sometimes called the position-wise feed-forward network, is the second main sub-layer within each Transformer block (following the multi-head attention sub-layer).

It consists of two linear transformations with a non-linear activation function (the GELU function we explored earlier) applied in between. The first linear layer usually expands the dimensionality of the input (e.g., from 768 to 3072 in GPT-2), the GELU activation introduces non-linearity, and the second linear layer projects the result back down to the original input dimensionality (e.g., 3072 back to 768).



This FFN is applied independently to each position in the sequence. While the attention mechanism allows tokens to interact with each other, the FFN processes the information at each position separately, transforming the representation based on the features learned by the model. It adds significant representational capacity to the model, allowing it to learn more complex functions beyond what the attention mechanism alone can capture.

```
# ffn:
#   x : (N, 768)
#   c_fc_w : (768, 3072)
#   c_fc_b : (3072,)
#   c_proj_w : (3072, 768)
```

```

# c_proj_b : (768,)
# out : (N, 768)
def ffn(
    x: np.ndarray,
    c_fc_w: np.ndarray,
    c_fc_b: np.ndarray,
    c_proj_w: np.ndarray,
    c_proj_b: np.ndarray,
) -> np.ndarray:
    return linear(gelu(linear(x, w=c_fc_w, b=c_fc_b)), w=c_proj_w, b=c_proj_b)

```

Attention

Attention is the core idea of the transformer architecture. It is a mechanism inspired by human visual attention. Just as we focus on certain parts of an image or specific words in a sentence while ignoring others, attention allows a model to dynamically weigh the importance of different parts of the input data when processing a particular element. For sequential data like text, this means that when the model is processing one word, it can "pay attention" to other relevant words in the sequence, regardless of their distance, to better understand the context and meaning. This selective focus helps the model capture long-range dependencies and relationships more effectively than older sequence models.

At the heart of each attention calculation are three key matrices derived from the input sequence:

Query (Q): Represents the current token or position we are processing. Think of it as asking: "What specific information am I looking for right now?"

Key (K): Represents all the tokens in the sequence (including the current one). Think of it as labels or identifiers for the information contained in each token: "What kind of information does each token offer?"

Value (V): Also represents all tokens in the sequence. Think of it as the actual content or representation of each token: "What is the actual information conveyed by each token?"

The process begins with an initial projection where the input sequence x passes through three separate linear layers to generate initial Q, K, and V matrices spanning the entire input dimension. These large matrices are then split into smaller chunks along their embedding dimension, creating separate Q, K, and V sets for each attention "head". Formally self-attention is computed as:

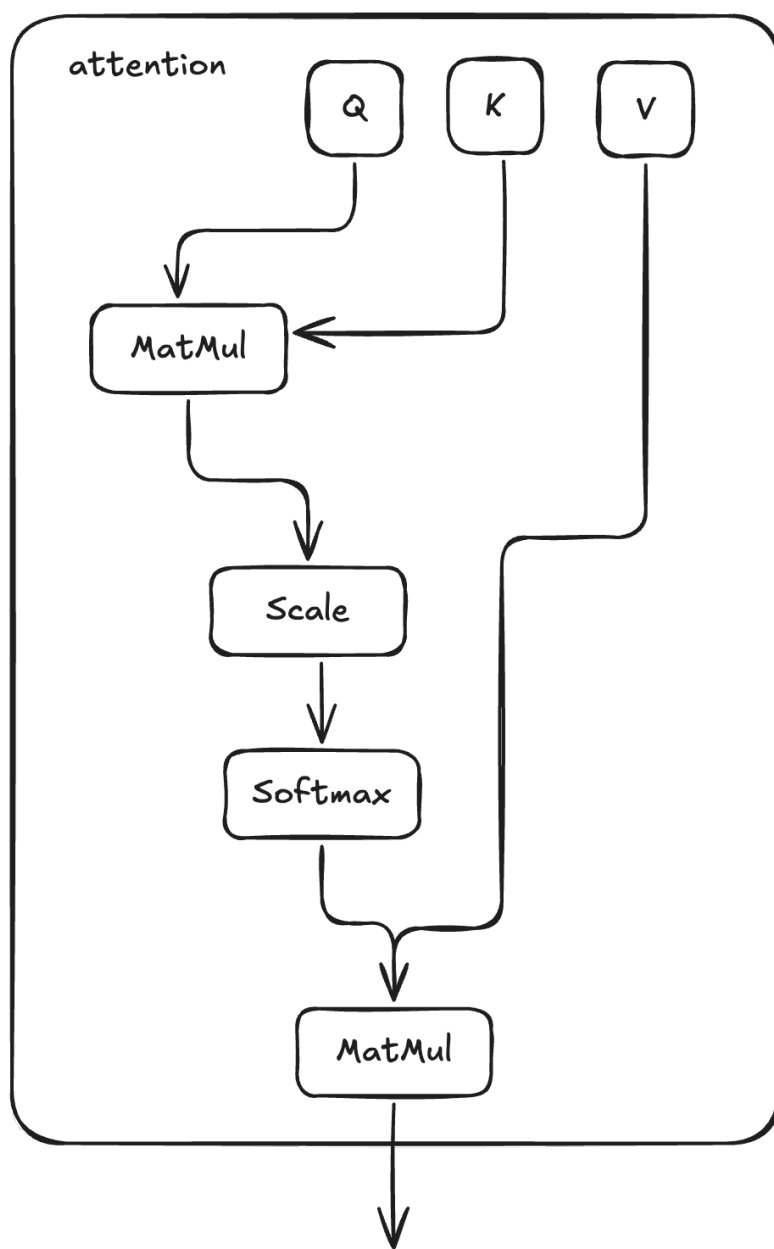
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

where:

Q, K, V are the query, key, and value matrices, and

d_k is the dimension of the key vectors.

The `attention` function implements the core scaled dot-product attention mechanism, the fundamental building block used within each head of the Multi-Head Attention layer. It calculates how much focus or "attention" each input token (represented by its Value vector) should receive when processing a specific token (represented by its Query vector).



The process starts by computing the dot product between the Q matrix and the transpose of the K matrix ($q @ k.T$). This step measures the raw similarity or compatibility between each query and all keys. These raw scores are then scaled down by dividing them by the square root of the dimension of the key vectors (`np.sqrt(q.shape[-1])`).

The causal mask is a mask that is added to these scaled scores. This mask prevents positions from attending to subsequent positions, ensuring predictions are based only on previous tokens. The mask effectively assigns very large negative values to scores corresponding to future tokens.

Next, a `softmax` function is applied to the masked, scaled scores. This converts the scores into probability distributions (attention weights) where each weight indicates the relative importance of a specific key (and its corresponding value) to the query.

Finally, the function computes the matrix product of these attention weights and the V matrix (`attention_weights @ v`). This yields the output of the attention mechanism: a weighted sum of the Value vectors, where the weights are the calculated attention probabilities. The resulting vector for each query is thus a blend of information from the input sequence, emphasizing the most relevant parts based on the query-key interactions.

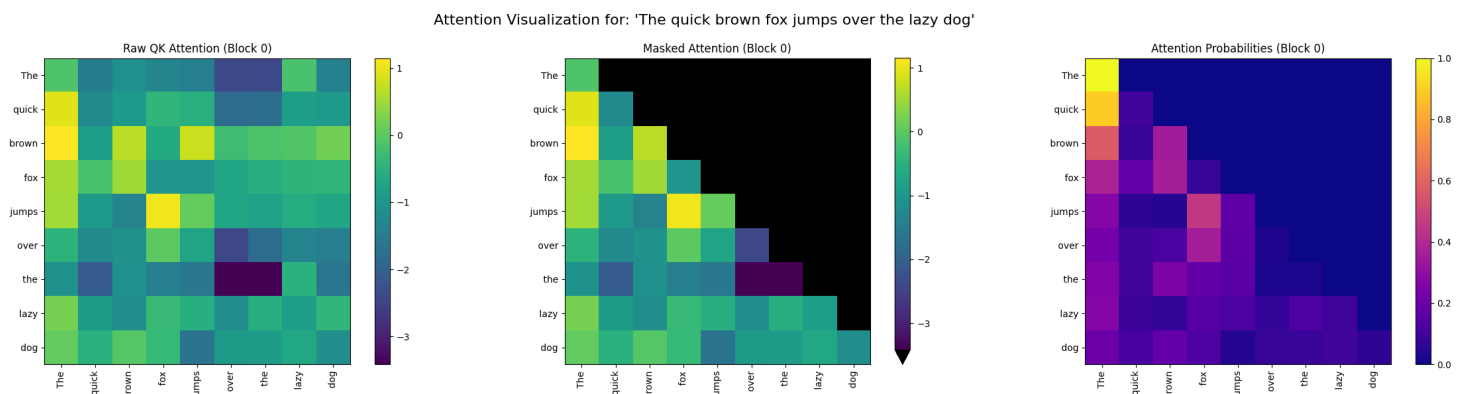
```
# attention:
#   q : (N, 64)
```

```

# k : (N, 64)
# v : (N, 64)
# mask : (N, N)
# out : (N, 64)
def attention(
    q: np.ndarray, k: np.ndarray, v: np.ndarray, mask: np.ndarray
) -> np.ndarray:
    attention_scores = (q @ k.T) / np.sqrt(q.shape[-1]) + mask
    attention_weights = softmax(attention_scores)
    return attention_weights @ v

```

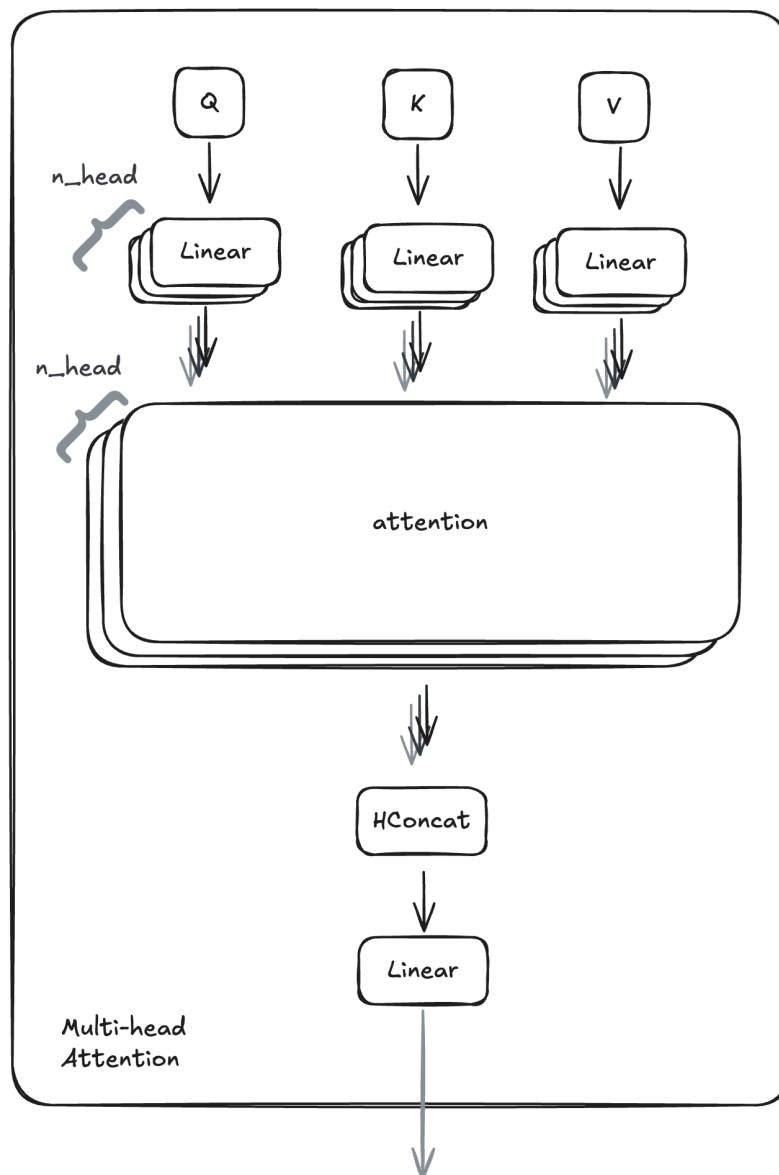
We can visualize the attention mechanism by plotting the attention weights for each head. For example for the phrase *"The quick brown fox jumps over the lazy dog"* we can plot the attention weights in the first block of the transformer stack to see which pairs of words are attended to most by each head. Here we see *"jumps"* and *"fox"* are attended to most by the first head.



Multi-Head Attention

While the basic attention mechanism provides a way to capture relationships between tokens, transformers take this a step further with Multi-Head Attention. This enhancement allows the model to capture different types of relationships simultaneously, greatly expanding its representational power. Multi-Head Attention is a core component that allows the model to jointly attend to information from different representation subspaces at different positions. Instead of performing a single attention calculation, MHA runs the attention mechanism multiple times ("heads") in parallel on different learned linear transformations of the input. This allows the model to capture various types of relationships (e.g., syntactic, semantic) simultaneously.

Within each head, scaled dot-product attention is performed. Attention scores are computed by taking the dot product of the Q with all Ks, measuring the relevance between the current token and every other token. These scores are scaled (divided by the square root of the key dimension) and passed through a softmax function to become probabilities (attention weights). These weights indicate how much focus to place on each V.



The size of each attention head's query matrix (64) is derived by dividing the model's embedding dimension (768) by the number of attention heads (12). When the outputs of all heads are concatenated back together, we recover the original embedding dimension:

$$768/n_head = 768/12 = 64$$

The attention weights are then used to multiply the corresponding V vectors, producing a weighted sum. This creates an output vector for the head that blends information from the most relevant tokens. Finally, the output vectors from all attention heads are concatenated back together. This combined vector is passed through a final linear layer to produce the overall output of the Multi-Head Attention block. This multi-faceted approach enables the model to capture richer contextual information by looking at the sequence from multiple "perspectives" concurrently.

```
# mha:
#   x : (N, 768)
#   out : (N, 768)
def mha(
    x: np.ndarray, c_attn: LinearParams, c_proj: LinearParams, n_head: int
) -> np.ndarray:
    # qkv projection
    # [N, 768] -> [N, 3*768]
    x = linear(x, w=c_attn.w, b=c_attn.b)
```

```

# split into qkv
# [N, 3*768] -> [3, N, 768]
qkv = np.split(x, 3, axis=-1)

# split into heads
# [3, N, 768] -> [3, n_head, N, 64]
qkv_heads = [np.split(x, n_head, axis=-1) for x in qkv]

# apply causal mask to hide future inputs
# [N, N]
causal_mask = (1 - np.tri(x.shape[0], dtype=x.dtype)) * -1e10

# perform attention on each head
# [3, n_head, N, 64] -> [n_head, N, 64]
out_heads = [attention(q, k, v, causal_mask) for q, k, v in zip(*qkv_heads)]

# merge heads
# [n_head, N, 64] -> [N, 768]
x = np.hstack(out_heads)

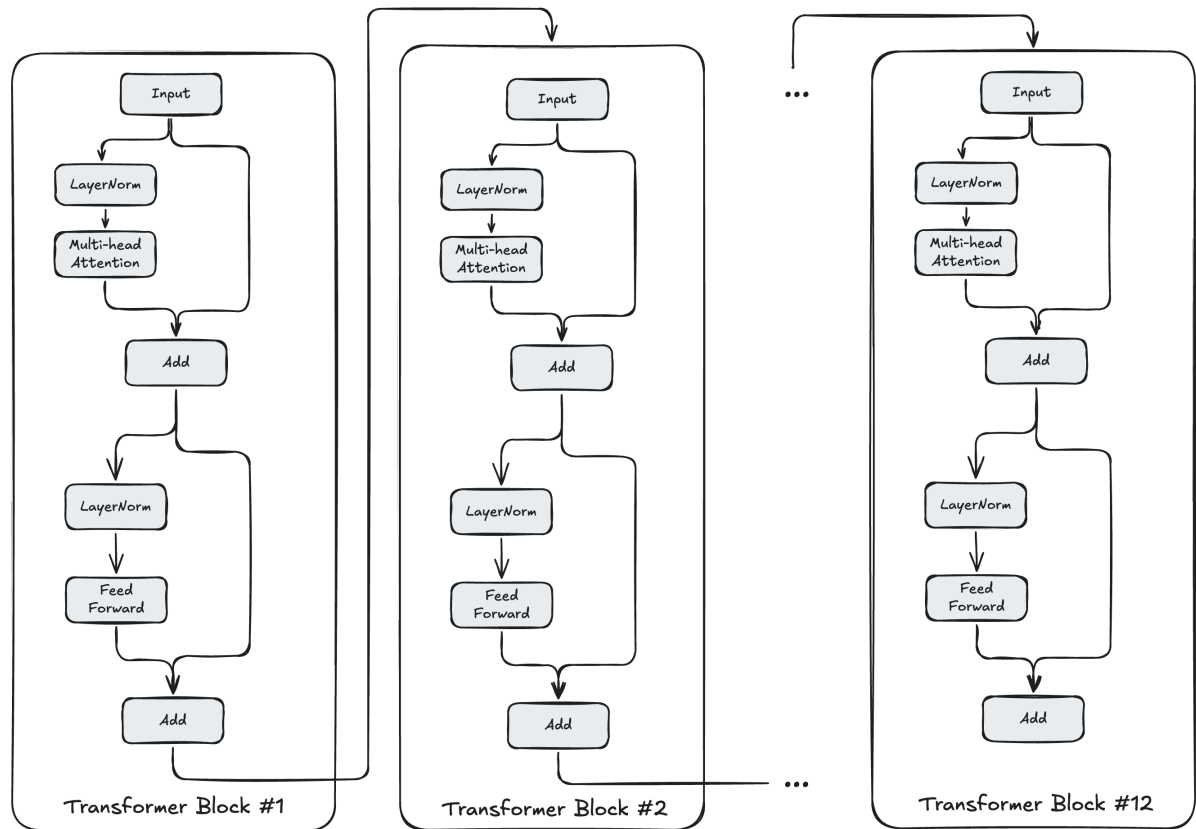
# out projection
# [N, 768] -> [N, 768]
return linear(x, w=c_proj.w, b=c_proj.b)

```

Transformer Block

With both the attention mechanism and feed-forward networks defined, we can now see how these components are integrated into the complete Transformer block. The Transformer block is the fundamental repeating unit of the GPT-2 architecture, combining multi-head attention with feed-forward processing, normalization, and residual connections to create a computational unit that can be stacked multiple times.

Each block takes a sequence of input vectors and processes them through two main sub-layers, incorporating residual connections and layer normalization around each. This structure creates a balanced flow of information that allows both preservation of original inputs (through residual connections) and transformation into new representations.



The first sub-layer is the Multi-Head Attention mechanism we just explored. Before the input enters the MHA, it undergoes layer normalization. The output of the MHA is then added back to the original input (a residual connection), combining the attention-derived context with the original positional representation. This sum then passes to the second sub-layer.

The second sub-layer is the Feed-Forward Network described earlier. Similar to the first sub-layer, the input (which is the output of the first residual connection) first goes through layer normalization. The normalized output is then processed by the FFN. The output of the FFN is then added back to its input via another residual connection. The final output of the transformer block is this sum, which then serves as the input to the next identical transformer block in the stack.

```
# transformer_block:
#  x : (N, 768)
#  out : (N, 768)
def transformer_block(
    x: np.ndarray,
    mlp: MLPParams,
    attn: AttentionParams,
    ln_1: LayerNormParams,
    ln_2: LayerNormParams,
    n_head: int
) -> np.ndarray:
    # First sub-block: Layer norm -> Attention -> Residual
    a = layer_norm(x, g=ln_1.g, b=ln_1.b)
    a = mha(a, c_attn=attn.c_attn, c_proj=attn.c_proj, n_head=n_head)
    x = x + a

    # Second sub-block: Layer norm -> FFN -> Residual
```

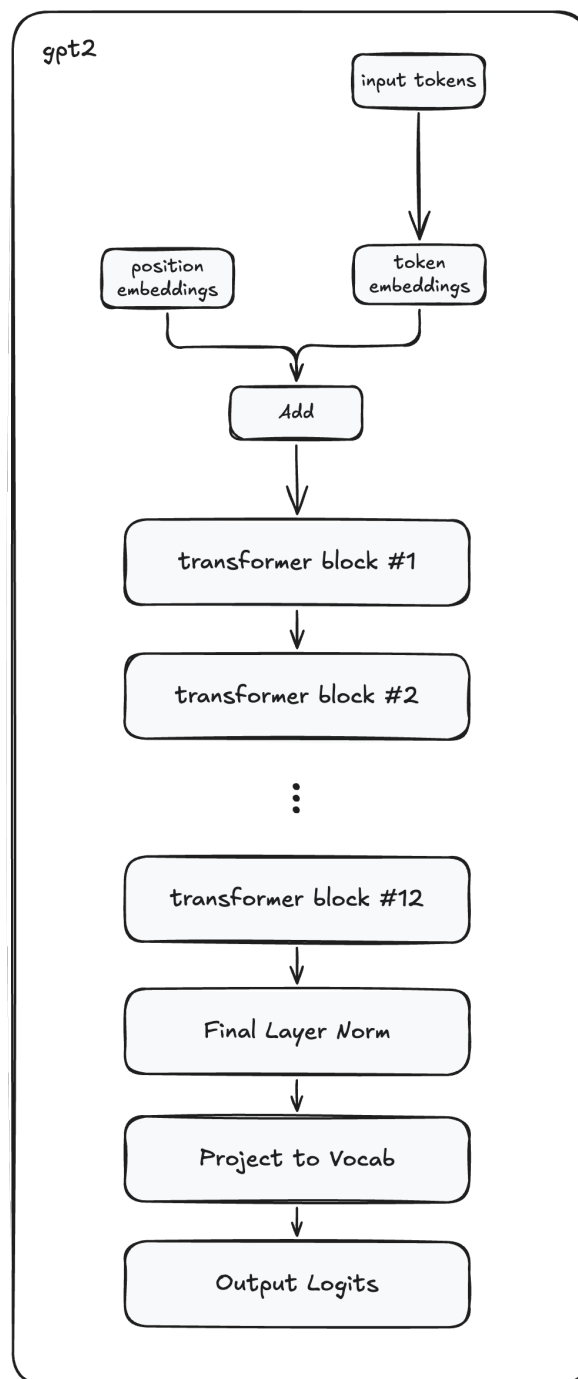
```
m = layer_norm(x, g=ln_2.g, b=ln_2.b)
m = ffn(
    m,
    c_fc_w=mlp.c_fc.w,
    c_fc_b=mlp.c_fc.b,
    c_proj_w=mlp.c_proj.w,
    c_proj_b=mlp.c_proj.b,
)
x = x + m

return x
```

Full Model

Now that we've examined the individual components and the Transformer block structure, we can assemble everything into a complete model. These blocks are stacked multiple times (e.g., 12 times in the smallest GPT-2 model) to build a deep network capable of sophisticated language understanding and generation. The combination of attention mechanisms, feed-forward networks, layer normalization, and residual connections creates an architecture that effectively processes sequential information, captures complex dependencies, and trains stably despite its depth.

The `gpt2` function represents the complete forward pass of the GPT-2 model for a given sequence of input token IDs. It orchestrates the flow of data through all the previously defined components, transforming raw token IDs into contextual representations and ultimately into predictions for the next token.



The journey begins with embedding the input tokens. This is achieved by summing the token embeddings (looked up in `params.wte`) and the positional embeddings (looked up in `params.wpe`) for each token in the input sequence. This initial combined embedding captures both the semantic meaning of each token and its position in the sequence, providing the foundation for all subsequent processing.

This embedded sequence then flows through the stack of Transformer blocks. Each block refines the representation through its attention and feed-forward mechanisms, gradually building more sophisticated contextual understanding. The code explicitly implements this stacking for a 12-layer model, where the output of one block becomes the input to the next. With each successive layer, the model can capture increasingly complex patterns and relationships in the data.

```
def gpt2(inputs: list[int], params: ModelParams, n_head: int) -> np.ndarray:
    # Get token embeddings and position embeddings
    x = params.wte[inputs] + params.wpe[range(len(inputs))]

    # Apply transformer block stack ( 12 blocks total )
    for i in range(12):
```

```

        x = transformer_block(
            x,
            n_head=n_head,
            mlp=params.blocks[i].mlp,
            attn=params.blocks[i].attn,
            ln_1=params.blocks[i].ln_1,
            ln_2=params.blocks[i].ln_2,
        )

# Apply final layer norm and project to vocabulary
x = layer_norm(x, g=params.ln_f.g, b=params.ln_f.b)
logits = x @ params.wte.T # Project to vocabulary

return logits

```

After processing through all the transformer blocks, the resulting sequence undergoes a final layer normalization to stabilize the values. This normalized output with shape $(N, 768)$ (for GPT-2 small) represents the final contextual embeddings for each token in the sequence. To convert these embeddings into predictions over the vocabulary, the model performs a projection using the transposed token embedding matrix (`params.wte.T`). This clever weight-sharing technique (using the same embedding matrix for both input embedding and output projection) reduces the total parameter count.

The result is a tensor of logits with shape $(N, 50257)$ (for GPT-2), where 50,257 is the vocabulary size. These logits represent the unnormalized prediction scores for each possible next token at each position in the sequence. Every token in the vocabulary receives a score, and these scores can be ranked to determine which tokens are most likely to follow in the sequence. A typical example of these logits for a single position might look like:

```

array([-114.83902718, -111.23177705, -116.58203861, ..., -118.4023539 ,
       -118.92616557, -113.37047973], shape=(50257,))

```

For autoregressive generation, we typically only care about predicting the next token after the end of our current sequence. By selecting the logits corresponding to the last position and applying an operation like `argmax`, we can identify the most likely next token:

```

next_token = np.argmax(logits[-1, :])

```

This token ID can then be appended to our sequence, and the process repeated to generate text one token at a time. But for this approach to work, we need properly initialized model parameters loaded from pre-trained weights, which brings us to our next topic.

Safetensors and Model Parameters

To utilize pre-trained models like GPT-2, we need to load and organize their parameters effectively. This section explains how we structure and load these parameters using the safetensors format, a secure and efficient method for storing model weights. For that we load them from HuggingFace in which they are stored in the `model.safetensors` file.

Safetensors is a fast and safe (i.e. it doesn't use Pickle, which requires execution of arbitrary code) format for storing tensors. The format uses a simple key/value structure where keys are UTF-8 encoded strings representing tensor names (e.g. `model.layers.0.attention.weight`), values are binary tensor data with a fixed header containing shape and dtype information, and a metadata section at the start of the file contains an index of all tensors and their offsets.

Internally the model parameters for GPT-2 are stored in the following key/value format:

```
{
  "wpe.weight"          : np.array([1024, 768]),
  "wte.weight"          : np.array([50257, 768]),
  ...
  "h.0.attn.bias"       : np.array([1, 1, 1024, 1024]),
  "h.0.attn.c_attn.bias" : np.array([2304]),
  "h.0.attn.c_attn.weight" : np.array([768, 2304]),
  "h.0.attn.c_proj.bias" : np.array([768]),
  "h.0.attn.c_proj.weight" : np.array([768, 768]),
  "h.0.ln_1.bias"       : np.array([768]),
  "h.0.ln_1.weight"     : np.array([768]),
  "h.0.ln_2.bias"       : np.array([768]),
  "h.0.ln_2.weight"     : np.array([768]),
  "h.0.mlp.c_fc.bias"   : np.array([3072]),
  "h.0.mlp.c_fc.weight" : np.array([768, 3072]),
  "h.0.mlp.c_proj.bias" : np.array([768]),
  "h.0.mlp.c_proj.weight" : np.array([3072, 768]),
  ...
  "ln_f.bias"           : np.array([768]),
  "ln_f.weight"         : np.array([768])
}
```

Each of these is a tensor with a shape and a dtype that hold the learned parameters for that component, these are the result of the model training process.

The `LayerNormParams` dataclass represents the layer normalization parameters, which include a scale (`g`) and bias (`b`) for each layer.

```
@dataclass
class LayerNormParams:
    g: np.ndarray # Gamma (scale)
    b: np.ndarray # Beta (bias)
```

The `LinearParams` dataclass represents the linear projection parameters, which include a weight matrix (`w`) and a bias vector (`b`).

```
@dataclass
class LinearParams:
    w: np.ndarray # Weight matrix
```

```
b: np.ndarray # Bias vector
```

The `MLPParams` dataclass represents the feed-forward network parameters, which include two linear layers: a first linear layer (`c_fc`) and a second linear layer (`c_proj`).

```
@dataclass
class MLPParams:
    c_fc: LinearParams # First linear layer
    c_proj: LinearParams # Second linear layer
```

The `AttentionParams` dataclass represents the attention parameters, which include a query-key-value projection (`c_attn`) and an output projection (`c_proj`).

```
@dataclass
class AttentionParams:
    c_attn: LinearParams # QKV projection
    c_proj: LinearParams # Output projection
```

The `TransformerBlockParams` dataclass represents a single transformer block, which includes a layer normalization (`ln_1`), a feed-forward network (`mlp`), an attention mechanism (`attn`), and a second layer normalization (`ln_2`).

```
@dataclass
class TransformerBlockParams:
    ln_1: LayerNormParams # First layer norm
    ln_2: LayerNormParams # Second layer norm
    mlp: MLPParams # MLP block
    attn: AttentionParams # Attention block
```

The `ModelParams` dataclass represents the complete model parameters, which include a token embedding matrix (`wte`), a position embedding matrix (`wpe`), a list of transformer blocks (`blocks`), and a final layer normalization (`ln_f`).

```
@dataclass
class ModelParams:
    wte: np.ndarray # Token embeddings
    wpe: np.ndarray # Position embeddings
    blocks: List[TransformerBlockParams] # Transformer blocks
    ln_f: LayerNormParams # Final layer norm
```

The `HParams` dataclass represents the hyperparameters of the model, which include the number of transformer layers (`n_layer`), the number of attention heads (`n_head`), and the context length (`n_ctx`).

```
@dataclass
class HParams:
    n_layer: int # Number of transformer layers
    n_head: int # Number of attention heads
    n_ctx: int # Context length
```

We can load the model parameters from a `model.safetensors` file using the `safe_open` function. This file

can be downloaded from the [Hugging Face Model Hub](#).

```
tensors = {}

with safe_open(weights_path, framework="numpy") as f:
    for key in f.keys():
        tensors[key] = f.get_tensor(key)

# Build transformer blocks
blocks = []

for i in range(config["n_layer"]):
    prefix = f"h.{i}"
    block = TransformerBlockParams(
        ln_1=LayerNormParams(
            g=tensors[f"{prefix}.ln_1.weight"], b=tensors[f"{prefix}.ln_1.bias"]
        ),
        ln_2=LayerNormParams(
            g=tensors[f"{prefix}.ln_2.weight"], b=tensors[f"{prefix}.ln_2.bias"]
        ),
        mlp=MLPParams(
            c_fc=LinearParams(
                w=tensors[f"{prefix}.mlp.c_fc.weight"],
                b=tensors[f"{prefix}.mlp.c_fc.bias"],
            ),
            c_proj=LinearParams(
                w=tensors[f"{prefix}.mlp.c_proj.weight"],
                b=tensors[f"{prefix}.mlp.c_proj.bias"],
            ),
        ),
        attn=AttentionParams(
            c_attn=LinearParams(
                w=tensors[f"{prefix}.attn.c_attn.weight"],
                b=tensors[f"{prefix}.attn.c_attn.bias"],
            ),
            c_proj=LinearParams(
                w=tensors[f"{prefix}.attn.c_proj.weight"],
                b=tensors[f"{prefix}.attn.c_proj.bias"],
            ),
        ),
    )
    blocks.append(block)

# Build final model params
params = ModelParams(
    wte=tensors["wte.weight"],
```

```
wpe=tensors["wpe.weight"],
blocks=blocks,
ln_f=LayerNormParams(g=tensors["ln_f.weight"], b=tensors["ln_f.bias"]),
)
```

Greedy Decoding

With the model architecture defined and parameters loaded, we're now ready to put everything together and generate text. The simplest approach for text generation is greedy decoding, where at each step we simply choose the token with the highest probability as predicted by our model.

The `generate` function below encapsulates the entire text generation process, from encoding the initial prompt to iteratively generating new tokens. This function showcases how all the components we've explored—tokenization, embedding, transformer blocks, and vocabulary projection—work together to create a functioning language model.

```
def generate(prompt: str, n_tokens_to_generate: int = 40) -> str:
    encoder = get_encoder("", "model")
    params, hparams = load_gpt2_weights()

    input_ids = encoder.encode(prompt)
    generated_token_ids: list[int] = []

    print(prompt, end="", flush=True)

    current_ids = list(input_ids)

    for _ in range(n_tokens_to_generate):
        logits = gpt2(current_ids, params, n_head=hparams.n_head)
        next_id = np.argmax(logits[-1, :])
        next_id_int = int(next_id)

        current_ids.append(next_id_int)
        generated_token_ids.append(next_id_int)

        token_text = encoder.decode([next_id_int])
        print(token_text, end="", flush=True)

    print()

    return encoder.decode(generated_token_ids)
```

The generation process starts by encoding the user's prompt into token IDs using our BPE tokenizer. We initialize an empty list to keep track of the tokens we'll generate. Then we enter a loop where, at each iteration, we:

Feed the current sequence of token IDs (the original prompt plus all tokens generated so far) through our GPT-2 model

Extract the logits for the last token position, which represent the model's prediction for what comes next

Take the argmax of these logits to select the most likely next token

Append this new token to our list of generated tokens and to the current sequence

Decode and print the new token to show the generation in real-time

After generating the requested number of tokens, we decode the full sequence of generated tokens back into text and return it. And that's it that's all it take to impmlement a naive version of a GPT-2 model. There's only about six "kernel" operations that we need to implement (`softmax`, `gelu`, `layer_norm`, `linear`, `add`, `matmul`, `var`, `mean`) and the rest is just a lot of bookkeeping to handle the data structures.

But as you notice, this model is *very* slow and inefficient averaging about 2-3 tokens per second on a modern CPU. There are many opportunities for both fusion and parallelization via lowering into MLIR and PTX, which we'll explore in the next section.

External Resources

[FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

[QLoRA: Efficient Finetuning of Quantized LLMs](#)

[The Illustrated GPT-2](#)

[The Annotated Transformer](#)

[attention-gym](#)